Rubrica: Systems Design

Learning a black box, one proven rule at a time · Reto 2 · La caja negra · rubrica.dev · Kevin Vicent · v6

> **Thesis:** The black box is an unknown decision procedure. We find its rules with experiments. We prove each rule against the real service. Then we compile the rules into a deterministic replica. Most agent systems need an LLM judge. Our system has an exact judge: the original service.

1. Problem

A legacy **ledger service** receives text messages on `POST /msg`. The body is `{"m": "CMD;arg…;control"}`. The commands are `PING`, `ALTA`, `CONSULTA`, `DEPOSITO`, `TRANSFER` and `ANULA`. The service also has `/__reset` and `/health`.

We must build a replacement. After `/__reset`, each response must have **the same status and body** as the original. The comparison ignores JSON key order and **ignores headers**. There are 100 hidden tests: 40 single requests, 45 sequences of 2–5 steps and 15 sequences of 6–12 steps. Every quirk is part of the contract (Hyrum's Law), for example `"recieved"`, `"comando desconocido "` and `cuentaId` vs. `cuenta_id`. The replica must not call the original. **The service changes during the day. The tests use the 20:00 version.**

Assumptions (we test each one): A1 The output depends only on state and request. **A2** Each token has its own state. `/__reset` restores the initial state (`AC-0001`, `OP-0001`). **A3** (*hypothesis*) The control is an **integer checksum on arguments only**. Observed values span 3–98; modular and polynomial formulas are candidate hypotheses. The service checks control **before** account lookup (bad control on missing account returns E202, not E401). **A4** Tests are sequential. **Hard cases:** the control formula; auth (team token, other token, no token); **error precedence** (E2xx integrity, E301/E302 syntax, E401 lookup, E506/E507 business rules); ID sequences; `ANULA` twice; limits; unknown routes; **drift**. **Out of scope:** headers (not tested); latency below the 5 s step limit.

2. Architecture: dual-process, one ontology

1. **Agent harness (System 2, offline).** A person and a reasoning model use only recorded tools: `probe`, `sequence`, `fuzz`, `shrink`, `diff`. Each request becomes one line of `experimentos.jsonl`, with its hypothesis and X–Req. **Rule: no request without a prediction.** A designed code gate (planned `beforeToolCall` hook) will block a request if it has no predicted response, if the log already predicts it, or if it exceeds the budget.
2. **Hypothesis search, not answer search (active learning).** Our first sweep tried controls 0–99 and used 543 requests. Now the harness keeps a set of candidate rules. It fits them offline on the log, at zero cost. Then it sends requests designed to maximally eliminate candidate families, targeting convergence within tens of targeted probes.
3. **Service ontology (memory).** Each rule is typed: `Rule{command, when:[ordered preconditions], then:{status, body}, effect, evidence:[log ids], confianza}`. The checksum is a typed slot: `Checksum{scope, mapping, base, modulus, transform}`. The same table does three jobs. The replica **executes** it. `SPEC.md` **is generated** from it (7 sections, `confirmado / probable / supuesto`). The tests come from its evidence.
4. **Replica (System 1, runtime).** HTTP adapter → serial queue → decision core `handle(state[token], msg) → (state, resp)`. Order (precedence under test): route → auth → parse → control → command → arity → lookup → business rule → effect. Control-before-lookup is proven; syntax vs integrity precedence is prioritized for verification. State is in memory.

Decision	Chosen	Rejected, and why
Runtime	**Compiled rule table**	*LLM in the replica (allowed, ≤ 5 s):* not exact, not repeatable. *Record and replay:* fails on new inputs.
Discovery	**Person + model in a harness**, exact oracle	*Free LLM chat:* no proof, no log. *Autonomous agent:* false spec claims **lose points**. *Manual curl:* too slow.
Knowledge	**Typed ontology**, one source for code, spec and tests	*Separate spec, code and tests:* they diverge after the change. *RAG over logs:* approximate recall.
Checks	**Code for all exact checks** (budget gate, checksum, rule replay). A **System One classifier** only for drift triage and model routing. It fails closed.	*LLM judge as verifier:* false passes. The original is a free, exact judge.
Server and hosting	**Minimal server we control** (exact bodies, also for unknown routes); **one always-on instance**	*Framework defaults:* their 404/405 bodies are different. *Autoscaling:* each instance has its own state.

3. How we measure: evals first

We optimize against evals and we protect them from Goodhart's Law: **corpus** = train, **held-out fuzz** = test, **canary** = "is the eval still true?", **hidden set** = final test.

- **M1 Exact match** with the official comparator. Target: 100% on the corpus (versioned per service version) and the 20 `ejemplos`; **≥ 95% on held-out fuzz**. We never fix code against held-out fuzz.
- **M2 Coverage:** command × precondition × error matrix. The fuzzer tests **low-confidence rules and empty cells first**. **Budget per token: 8,000 requests/day, 10/s.** Plan (deducting 565 spent): 1,435 discovery, 2,500 fuzz, 500 canary (12:00–20:00), 3,000 reserve. A teammate's token adds 8,000.
- **M3 Drift:** a **10-request canary** every 10 min (480 requests across the 12:00–20:00 window). A full replay runs only if the canary fails. Detect in ≤ 10 min. Fix in ≤ 45 min.
- **Checksum eval:** the formula must predict all logged controls (train) and new strings with digits, lowercase and accents (test).
- **Seeded tests:** each error alone and **in pairs** (precedence), control ±1, case variants, arity ±1, `ANULA` twice, transfers near the limit.
- **3 validations:** only if M1 did not decrease (~13:30, ~17:30, ~19:30).

Risks: the formula is not in our family (add candidates, eliminate by targeted probes) · budget runs out (cache, never repeat) · late drift (canary runs until 20:00).